FC 프로젝트 AI 개발 아키텍처 종합 분석 보고서

AGENTS.md 최상위 헌장, 도메인 룰(Rules), 자율 스킬(Skills), MCP 도구 및 순환 엔지니어링 체계 심층 해설

TARGET ENGINE	ARCHITECTURE CORE	AI AGENT ENVIRONMENT	REPORT DATE & VERSION
Unreal Engine 5.8 (Win64 / C++20)	Listen Server (SP = MP Parity)	Antigravity Autonomous Engine	2026-10-15 • Version 1.2

1. 총괄 요약 (Executive Summary)

본 문서는 **FC 프로젝트(Unreal Engine 5.8)**에 구축된 **Antigravity AI 페어 프로그래밍 및 자율 에이전트 개발 체계**의 기술적 구성 요소(`AGENTS.md`, `Rules`, `Skills`, `MCP`, `Scripts`)를 체계적으로 분석하여 정리한 공식 아키텍처 보고서입니다. 본 시스템은 단순한 프롬프트 전달 수준을 넘어, AI 에이전트가 엄격한 멀티플레이어 네트워크 권한을 준수하고, 프로젝트 전용 툴체인(UBT, 정적 분석기, 밸런스 시뮬레이터, 자동화 테스트)과 직접 연동하여 **최대 5회 자가 치유(Self-Healing)**를 수행할 수 있는 완결된 엔지니어링 루프를 제공합니다.

FC 프로젝트 AI 협업 환경 3대 핵심 설계 철학

CORE TENETS

- **Single-Player is Local Multiplayer:** 모든 게임플레이 시스템은 네트워크 경계를 전제로 설계됩니다. 단독 플레이도 엔진 내부 Listen Server 세션으로 동작하여 싱글과 협동/멀티플레이 간 100% 기능 동일성을 보장합니다. 클라이언트 임의 상태 변이 숏컷 작성을 엄격히 금지합니다.
- **Minimal Surface Area & Structural Preservation (최소한의 수정 & 기존 코드 활용):** 기존 클래스, 유틸리티, 데이터 테이블을 최대한 재활용하며, 변경 범위를 작업 요청에만 국한하여 불필요한 리팩토링이나 코드 훼손을 엄격히 방지합니다.
- **Token-Efficient Dual-Track Workflow:** 일상적인 코드 작성 및 Q&A 시에는 스크립트 실행 오버헤드가 없는 `Fast Track`을 유지하고, 검증 요청 시에만 단일 명령어(`pipeline.py` / `MCP`)로 통합 검증을 구동하여 AI 컨텍스트 토큰을 극대화합니다.



2. 최상위 아키텍처 헌장: AGENTS.md 심층 분석

`AGENTS.md`는 FC 프로젝트 리포지토리의 최상위 루트에 위치하며, **AI 에이전트 시스템에 최우선 규칙(Supreme Rule)**으로 자동 주입되는 핵심 헌장입니다. 모든 AI 코딩 작업은 이 문서에 기술된 권한 분리 원칙과 루프 엔지니어링 수칙을 절대적으로 준수해야 합니다.

가. 4계층 권한 및 관심사 분리 모델 (4-Tier Authority Model)

네트워크 멀티플레이어 환경에서 클라이언트 치팅 방지와 시각적 부드러움을 동시에 달성하기 위해 코드를 엄격히 4계층으로 격리합니다:

계층 (Tier)	대표 클래스 및 주체	주요 책임 및 실행 영역	보장 수칙 및 가드
1. Server Authority SERVER ONLY	`AGameModeBase`, `AAIController`, Authoritative Managers	전투 판정, 데미지 연산, 사망 처리, 인벤토리/재화/퀘스트 변이, 액터 스폰, Server RPC 인자 검증	`if (HasAuthority())` 또는 `ROLE_Authority` 필수 가드
2. Replicated State SERVER TO CLIENT	`AGameStateBase`, `APlayerState`, `UActorComponent`	게임플레이 상태 동기화(체력, 마나, 버프, 팀 점수), 조건부 복제 필터링, 변경 알림 발송	`DOREPLIFETIME_CONDITION` `OnRep_*` 콜백 핸들러
3. Autonomous Client CLIENT PREDICTION	`APlayerController`, `ACharacter` (Autonomous)	Enhanced Input 처리, 로컬 캐릭터 이동 예측, 사용자 UI 조작 의도 서버 전송	`Server_*` RPC 필수 `WithValidation` 의무화
4. Simulated Client & UI PRESENTATION TIER	`AHUD`, `UUserWidget`, `UNiagaraComponent`, Audio	UI 렌더링, HUD 갱신, SFX/VFX 재생, 랙 보간 연출 (서버 코드 종속성 0%)	서버 코드 내 Slate/UMG 인클루드 완전 금지

나. 최소 수정 및 구조 보존 원칙 (Minimal Disruption Directive)

> **AI 에이전트 코드 수정의 3대 안전 수칙** SAFETY RULES
>
> 1. **최소한의 수정 범위 (Minimal Surface Area Changes):** 요청된 작업과 무관한 파일, 함수, 공백, 서식을 임의로 재작성하거나 리팩토링하지 않으며, 가장 작고 명확한 기능적 Diff를 생성합니다.
> 2. **기존 구조 및 코드 재활용 극대화 (Maximize Reuse):** 새로운 추상화 계층이나 중복 함수를 임의로 만들기 전에 프로젝트 내 기존 클래스, 유틸리티 함수, 데이터 테이블, 서브시스템을 우선 조사하여 상속 및 확장합니다.
> 3. **문서 및 의도 무결성 유지 (Preserve Context & Docstrings):** 기존 주석, TODO 항목, 설계 의도를 삭제하지 않고 유지합니다.

다. 계층형 파이프라인 프로토콜 (Tiered Pipeline Protocol)

AI 에이전트의 불필요한 스크립트 실행과 토큰 낭비를 방지하기 위해 작업 모드를 엄격히 구분합니다:

- **기본 모드 (Fast Track):** 일반 코드 편집, 질문 답변, 간단한 리팩토링 시에는 검증 스크립트를 자동 실행하지 않고 깔끔한 C++ 코드와 설명을 즉시 제공합니다.
- **온디맨드 검증 모드 (On-Demand Single Tool Call):** 사용자가 컴파일이나 검증을 요청했을 때만 통합 마스터 러너(`pipeline.py`)를 단일 도구 호출로 실행합니다.

3. 모듈형 도메인 규칙 체계 (.antigravity/rules/)

모든 규칙을 상시 프롬프트에 담을 경우 발생하는 **컨텍스트 낭비와 환각(Hallucination)**을 방지하기 위해, 세부 구현 표준은 `.antigravity/rules/` 디렉토리에 6개의 모듈형 문서로 분리되어 필요 시점에만 **온디맨드(On-Demand)**로 참조됩니다.

01-ue58-coding-standards.md

C++20 & UE5.8

- **현대 C++20 표준:** `/std:c++20` 채택, 엔진 표준 컨테이너(`TArray`, `TMap`) 사용, 컴파일 타임 `constexpr` 활용.
- **스마트 포인터 위생:** `UPROPERTY` 멤버 변수에는 생포인터(`T*`) 대신 반드시 `TObjectPtr<T>` 강제, 캐시 참조는 `TWeakObjectPtr<T>`.
- **IWYU (Include What You Use):** 헤더에는 전방 선언(Forward Declaration)만 사용하고, `.generated.h`는 헤더의 가장 마지막에 인클루드.
- **표준 네이밍 접두사:** `A`(Actor), `U`(Object), `F`(Struct), `E`(Enum), `I`(Interface), `b`(Boolean).

02-multiplayer-conventions.md

NETWORKING

- **리플리케이션 수칙:** `Net/UnrealNetwork.h` 포함, `GetLifetimeReplicatedProps` 내 `Super` 호출 필수.
- **조건부 복제 의무화:** 무조건 `DOREPLIFETIME` 금지. `COND_OwnerOnly`, `COND_SkipOwner`, `COND_SimulatedOnly` 필터링 적용.
- **Server RPC 검증:** 모든 서버 RPC는 `WithValidation` 지정 및 `_Validate` / `_Implementation` 구현 쌍 작성.
- **Multicast RPC 제한:** 게임플레이 상태 변이 금지, 일회성 시각/음향 연출 전용.

03-loop-engineering.md

SELF-HEALING

- **자율 자가 치유 루프:** 컴파일 에러 발생 시 사용자 개입 없이 JSON 진단 데이터를 파싱하여 즉시 수정 코드 작성.
- **최대 5회 교정 예산:** 태스크당 최대 5회 반복 시도 후 지속 실패 시 원인 가설 및 에러 상세 리포트 제출.
- **에러 트리아지 가이드:** `C2065`(헤더 누락), `WithValidation` 누락, `LNK2019`(Build.cs 모듈 누락), UI 헤더 오염 분류.

04-gameplay-ability-system.md

GAS & CUE

- **ATTRIBUTE_ACCESSORS 매크로:** Getter, Setter, Init, Capture 표준 매크로 강제 적용.
- **어트리뷰트 복제:** `DOREPLIFETIME_CONDITION_NOTIFY`에 `REPNOTIFY_Always` 지정 및 `GAMEPLAYATTRIBUTE_REPNOTIFY` 콜백 사용.
- **클램핑 수칙:** `PreAttributeChange`(사전 클램프) vs `PostGameplayEffectExecute`(서버 권한 후처리).
- **Gameplay Cue:** 타격 이펙트/사운드에 멀티캐스트 RPC 대신 네이티브 거리 컬링을 지원하는 Cue 활용.

05-combat-and-inventory-networking.md

FAST ARRAY

- **FFastArraySerializer:** 대규모 동적 아이템 및 무기 목록 동기화 시 `TArray` 단순 복제를 금지하고 델타 직렬화 패턴 강제.
- **인벤토리 대역폭:** 컨테이너 프로퍼티를 `COND_OwnerOnly`로 등록하여 타 플레이어 불필요 패킷 전송 차단.
- **서버 히트 검증:** 클라이언트의 데미지 수치/좌표 불신. 서버가 FOV 각도, 거리, 레이캐스트/스윕을 통해 유효성 직접 검증.

06-umg-mvvm-standards.md

UMG MVVM

- **클라이언트 전용 뷰모델:** `UMVVMViewModelBase` 상속, 뷰모델 복제 금지(OnRep/델리게이트 수신 후 로컬 주입).
- **Zero-Tick 바인딩:** `Tick` 감시 제거, `FieldNotify` 브로드캐스트 기반 UI 이벤트 트리거.
- **변이 매크로:** 값 변경 감지 시 `UE_MVVM_SET_PROPERTY_VALUE`, 컬렉션 갱신 시 `UE_MVVM_BROADCAST_FIELD_VALUE_CHANGED`.
- **소프트 에셋 참조:** 아이콘/텍스처는 `TSoftObjectPtr<UTexture2D>`로 동기 블로킹 방지.

4. 자율 스킬 체계 (.antigravity/skills/)

Antigravity의 **Skill(스킬)**은 에이전트가 특정 도메인 태스크를 능동적으로 수행할 수 있도록 지침(Instructions), 실행 커맨드(Commands), 리소스(Resources)를 하나로 묶은 최신 에이전트 확장 규격입니다. FC 프로젝트는 언리얼 엔진 5.8 전용 빌드 및 검증 하네스를 `ue5-harness` 스킬로 패키징하여 에이전트의 자율 엔지니어링을 뒷받침합니다.

📦 ue5-harness 워크스페이스 스킬 구성

WORKSPACE SKILL V1.1.0

파일 경로: 메인 지침 `.antigravity/skills/ue5-harness/SKILL.md` | 커맨드 정의 `.antigravity/skills/ue5-harness-skill.json`

에이전트는 터미널에 긴 인자열을 직접 조합하는 대신, 사전 정의된 스킬 액션을 호출하여 일관되고 격리된 환경에서 언리얼 엔진 툴체인을 구동합니다.

스킬 액션명	매핑 실행 명령어	수행 작업 및 반환 데이터
`ue5_build`	`python Scripts/build_harness.py --pretty`	UBT(UnrealBuildTool) 컴파일 구동, C2xxx/LNKxxxx/UHT 에러 파싱 및 구조화된 JSON 반환
`ue5_verify_replication`	`python Scripts/verify_replication.py --pretty`	`Source/` 디렉토리 정적 분석, `WithValidation` 누락, `bReplicates` 누락, UI 헤더 오염 탐지
`ue5_verify_gas`	`python Scripts/verify_gas.py --pretty`	`UAttributeSet` 내 `ATTRIBUTE_ACCESSORS` 매크로, `OnRep_*` 콜백 및 GAS 규격 준수 여부 검사
`ue5_audit_bandwidth`	`python Scripts/audit_bandwidth.py --pretty`	원시 `TArray` 복제 안티패턴, 미필터링 `DOREPLIFETIME`, 대용량 스트링 복제 탐지
`ue5_simulate_ttk`	`python Scripts/simulate_ttk_balance.py --pretty`	무기 아키타입별 유효 DPS, 아머 티어별 TTK(Time-to-Kill) 커브 수학적 시뮬레이션 및 벤치마크
`ue5_simulate_net_pie`	`python Scripts/simulate_net_pie.py --pretty`	헤드리스 Dedicated Server/Client 구동, 패킷 지연(lag), 손실(loss), 복제(dup) 인젝션 테스트
`ue5_run_tests`	`python Scripts/run_tests.py --pretty`	`UnrealEditor-Cmd.exe` 헤드리스 자동화 스펙(NullRHI 모드) 실행 및 단위 테스트 상태 리포트

🌐 Antigravity 글로벌 Built-in 스킬과의 역할 분담

SKILLS ECOSYSTEM

- `agy-customizations` / `antigravity-guide`: Antigravity 커스터마이징 메커니즘, 우선순위, 규칙 로딩, IDE 환경 전반을 안내하는 시스템 레벨 스킬.
- `automation`: 백그라운드 타이머 및 주기적 cron 자동화 작업을 설계하고 스케줄링하는 범용 자동화 스킬.
- `generative_ui`: 채팅 인터페이스 내에 반응형 대화식 HTML 위젯이나 다이어그램을 렌더링하는 시각화 스킬.
- `ue5-harness` **(프로젝트 전용):** FC 리포지토리에 종속된 특화 스킬로, UE 5.8 C++ 빌드와 멀티플레이 정적 검증 파이프라인을 전담.

5. Model Context Protocol (MCP) 연동 (.antigravity/mcp.json)

Model Context Protocol (MCP)은 LLM 에이전트와 로컬 개발 환경 간의 표준화된 도구 호출 인터페이스입니다. 터미널 명령어를 직접 파싱하는 수고 없이, 에이전트가 사전 정의된 구조적 도구(Tool)를 함수 호출하듯 실행하고 그 결과를 컨텍스트로 수신합니다.

.antigravity/mcp.json 서버 아키텍처

MCP SERVER: UE5-HARNESS

서버 실행 정의: `command: "python"`, `args: ["Scripts/build_harness.py"]`, `env: {"PYTHONUNBUFFERED": "1"}`

MCP 설정 파일에 정의된 `tools` 블록은 에이전트에게 7개의 전용 네이티브 함수로 노출됩니다. 각 도구는 `--pretty` 플래그를 통해 포맷된 JSON 결과를 스트림으로 반환하여 에이전트의 에러 분석 속도를 극대화합니다.

```
// .antigravity/mcp.json 서버 및 도구 등록 구조
{
  "mcpServers": {
    "ue5-harness": {
      "command": "python",
      "args": ["Scripts/build_harness.py"],
      "env": { "PYTHONUNBUFFERED": "1" },
      "tools": {
        "ue5_build": { "command": "python Scripts/build_harness.py --pretty", "description": "Compile UE5.8 project with UE
        "ue5_verify_replication": { "command": "python Scripts/verify_replication.py --pretty", "description": "Verify repl
        "ue5_verify_gas": { "command": "python Scripts/verify_gas.py --pretty", "description": "GAS compliance inspection"
        "ue5_audit_bandwidth": { "command": "python Scripts/audit_bandwidth.py --pretty", "description": "Scan network anti
        "ue5_simulate_ttk": { "command": "python Scripts/simulate_ttk_balance.py --pretty", "description": "Execute TTK sim
        "ue5_simulate_net_pie": { "command": "python Scripts/simulate_net_pie.py --pretty", "description": "Net degradation
        "ue5_run_tests": { "command": "python Scripts/run_tests.py --pretty", "description": "Headless automation test spec
      }
    }
  }
}
```

MCP 도구 호출 방식과 일반 Shell 실행 방식 비교 우위

COMPARISON MATRIX

비교 지표	기존 임의 Shell 실행 (PowerShell/Bash)	MCP 프로토콜 기반 도구 호출
구문 안정성	따옴표, 이스케이프 문자 오류, 윈도우 경로 역슬래시 결함 빈발	구조화된 인자 전달로 구문 에러율 0% 보장
토큰 효율성	수천 라인의 원시 컴파일러 로그가 컨텍스트 윈도우 잠식	`--pretty` JSON 요약본만 수신하여 토큰 90% 이상 절감
에러 트리아지	에이전트가 텍스트 정규식을 직접 돌려 파일과 라인 추론	`file`, `line`, `code`가 정형 JSON 키로 즉시 제공
보안 및 권한	임의 시스템 명령 실행 가능 위험성 존재	사전 승인된 화이트리스트 검증 스크립트만 격리 구동

6. 통합 파이프라인 및 순환 엔지니어링 (Unified Pipeline & Self-Healing)

여러 도구를 개별 호출하면서 발생하는 컨텍스트 윈도우 오버헤드와 왕복 지연을 방지하기 위해, FC 프로젝트는 단일 마스터 러너인 `Scripts/pipeline.py`와 이를 감싸는 터미널 래퍼(`pipeline.bat` / `pipeline.ps1`)를 운영합니다.

가. 파이프라인 실행 모드 체계

모드 명칭	실행 명령어	소요 시간	동작 내용 및 검증 단계
Fast Mode	(스크립트 미실행 - 기본 상태)	0.0s	단순 코드 작성, 설명, 리팩토링 시 스크립트를 일절 실행하지 않고 즉시 코드 제공 (토큰 보존)
static	`python Scripts/pipeline.py static`	~0.4s	GAS 준수, 대역폭 누수, 리플리케이션 위반 초고속 AST 정적 분석 (초저비용 토큰)
build	`python Scripts/pipeline.py build`	~5s - 30s	UBT 컴파일 구동, C++20/UHT 에러 JSON 트리아지, 에이전트 자가 치유 루프(최대 5회) 연동
balance	`python Scripts/pipeline.py balance`	~0.2s	전투 수학 모델링, 방어구 관통 공식 기반 TTK 및 유효 DPS 밸런스 지표 산출
test	`python Scripts/pipeline.py test`	~10s	헤드리스 언리얼 에디터 환경에서 Core Combat 및 AI 유닛 테스트 자동 실행
full	`python Scripts/pipeline.py full --auto-commit`	순차 실행	Static 검증 → Build 하네스 → Test 검증 순차 통과 시 Git 자동 커밋 생성

나. 자율 자가 교정(Self-Healing) 엔지니어링 프로세스

🔄 에러 자동 파싱 및 최대 5회 자가 치유 루프 동작 메커니즘

AUTONOMOUS LOOP

1. **에이전트 코드 수정:** C++ 소스 또는 헤더 파일 작성.
2. **검증 호출:** `pipeline.py build` 또는 `pipeline.py static` 단일 호출.
3. **JSON 파싱 & 진단:** 에러 발생 시 UBT 또는 AST 스크립트가 표준 JSON(`file`, `line`, `code`, `message`) 출력.
4. **원인 추론 및 코드 패치:** 사용자 개입 없이 에이전트가 해당 라인 전후 컨텍스트를 읽고 즉각 패치 적용.
5. **루프 반복 (Max 5 Cycles):** 정상 통과할 때까지 최대 5회 재컴파일 수행. 5회 초과 실패 시에만 정밀 원인 분석 리포트 제출 후 사용자에게 피드백 요청.

다. 에러 코드별 자동 해결 전략 가이드 (Triage Matrix)

에러 유형 및 코드	주요 발생 원인	에이전트 자가 치유 전략
C2065: undeclared identifier	타입 누락, 필수 헤더 미인클루드, 오타	해당 심볼 정의 위치 조사 후 `.cpp`에 `#include` 추가 또는 `.h`에 전방 선언 추가
UHT: Missing WithValidation	`Server` RPC에 `WithValidation` 지정 누락	`UFUNCTION` 매크로에 `WithValidation` 추가 및 `_Validate` 함수 구현
UHT: Missing LifetimeProps	`Replicated` 변수 선언 후 등록 누락	`GetLifetimeReplicatedProps` 내 `DOREPLIFETIME_CONDITION` 등록
LNK2019 / LNK2001	모듈 종속성 누락 또는 함수 본체 미구현	`FC.Build.cs`의 `PublicDependencyModuleNames`에 모듈 추가 (예: `NetCore`, `GameplayAbilities`)
UI 오염 (Replication Audit Fail)	서버 클래스에 Slate/UMG 위젯 헤더 포함	위젯 헤더 인클루드를 클라이언트 전용 클래스로 이동 및 델리게이트 이벤트 구조로 리팩토링

7. 구성 요소 간 상호작용 및 실무 개발 가이드라인

프로젝트 내 AI 개발 지원 요소들은 상호 유기적으로 결합되어 다음과 같은 책임 체계를 형성합니다:

구성 요소	물리 파일 위치	역할 및 권한 범위	상호작용 연계점
AGENTS.md	`/AGENTS.md` (Root)	시스템 최상위 헌장: 4계층 권한, 파이프라인 모드, 최소 수정 원칙 정의	모든 AI 세션에 기본 주입되며, Rules와 MCP를 총괄 지휘하는 기준점
Rules	`.antigravity/rules/*.md`	C++20, 리플리케이션, GAS, 인벤토리, MVVM 도메인별 세부 구현 수칙	작업 대상 기능에 따라 온디맨드로 참조되어 프롬프트 팽창 방지
Skills	`.antigravity/skills/ue5-harness/`	에이전트가 UBT 및 엔진 검증 스크립트를 호출할 수 있는 커맨드 패키지	에이전트 액션 인터페이스를 제공하며 `mcp.json`과 1:1 기능 정렬
MCP	`.antigravity/mcp.json`	LLM 컨텍스트와 로컬 파이썬 검증 스크립트를 연결하는 프로토콜 서버	`Scripts/` 내 개별 진단 스크립트들을 JSON 출력 툴로 노출
Pipeline Scripts	`Scripts/pipeline.py` 외 다수	실제 AST 파싱, UBT 빌드, 언리얼 에디터 헤드리스 실행을 담당하는 백엔드 엔진	MCP 툴 및 터미널 래퍼(`pipeline.bat/ps1`)의 실제 구동 엔진

💡 개발 상황별 실무 적용 가이드라인 (Developer & Agent Playbook)

ACTION GUIDE

- **일반 로직 및 C++ 클래스 수정:** `AGENTS.md`와 `01-ue58-coding-standards.md`를 준수하여 작성하며, 별도 검증 명령 없이 Fast Mode로 신속히 완료합니다.
- **멀티플레이어 액터/컴포넌트 작성:** `02-multiplayer-conventions.md`에 따라 `DOREPLIFETIME_CONDITION` 및 Server RPC `WithValidation`을 작성한 뒤 `pipeline.py static`으로 정적 검증합니다.
- **스킬 및 상태이상 추가:** `04-gameplay-ability-system.md`를 참조하여 `ATTRIBUTE_ACCESSORS`와 `GameplayCue`를 적용하고, `pipeline.py static --gas-only`로 검증합니다.
- **인벤토리/아이템 목록 추가:** `05-combat-and-inventory-networking.md`를 참조하여 `FFastArraySerializer`와 `COND_OwnerOnly`를 구현합니다.
- **UI 개발:** `06-umg-mvvm-standards.md`에 따라 서버 독립적인 `UMVVMViewModelBase` 및 `FieldNotify` 바인딩을 구축합니다.
- **커밋 전 최종 릴리즈:** `python Scripts/pipeline.py full --auto-commit -m "feat: ..."` 단일 명령어로 정적 검증, 컴파일, 유닛 테스트를 일괄 통과시킵니다.

🏁 결론 및 총평 (Conclusion)

PRODUCTION READY

FC 프로젝트의 AI 개발 아키텍처는 **최상위 헌장(AGENTS.md)**의 엄격한 권한 분리와 최소 침습 통율 하에, **모듈형 도메인 룰(Rules)**의 온디맨드 주입, **스킬(Skills)**과 **MCP**를 매개로 한 헤드리스 툴체인 연동, 그리고 **자가 치유 루프(Loop Engineering)**가 완벽한 삼위일체를 이루고 있습니다. 이는 AAA급 언리얼 엔진 5.8 멀티플레이어 환경에서 발생하기 쉬운 네트워크 데싱크, UBT 컴파일 오류, UI 서버 오염을 사전에 원천 차단하며, 극대화된 개발 속도와 코드 품질을 동시에 보장합니다.